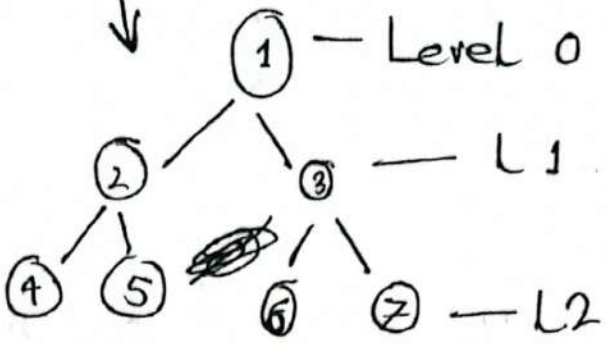
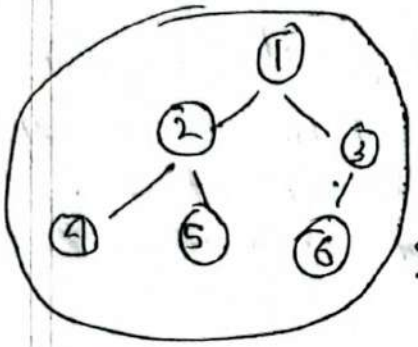
Online
17-10-26

Heap

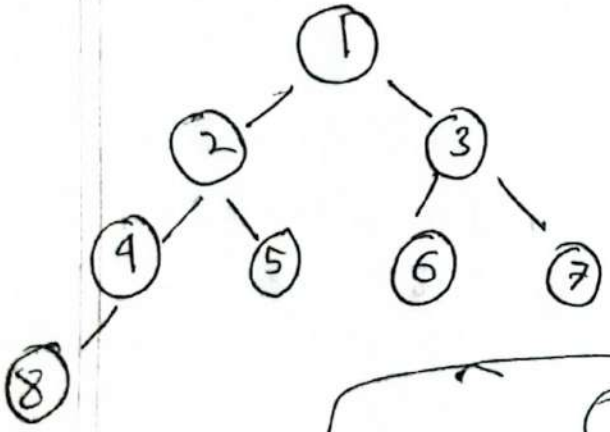
Compelete Binary tree (CBT)



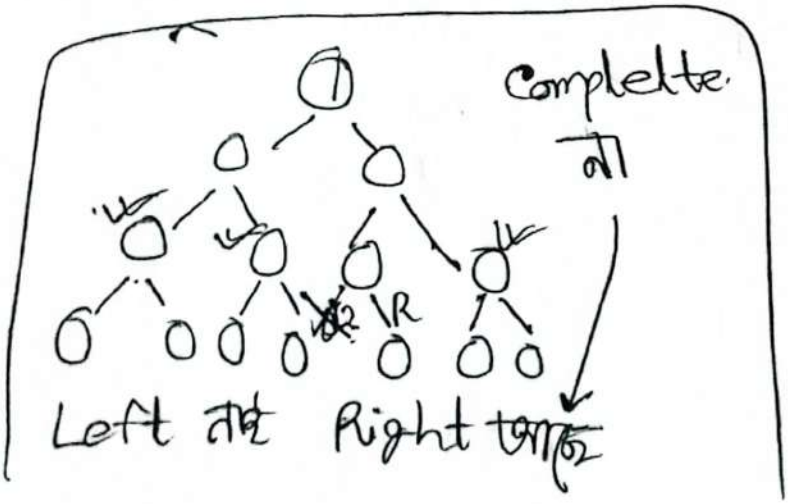


- complet

Last Level এ ডান দিকে কোনো Absent থাকতে পারে।

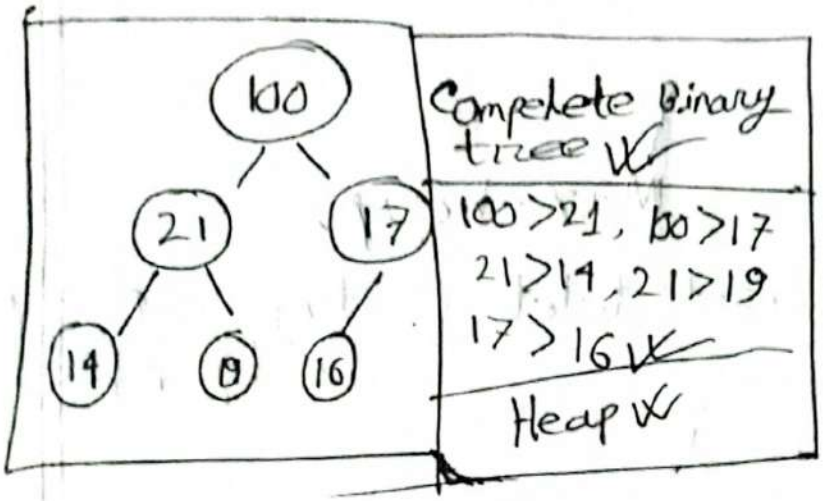


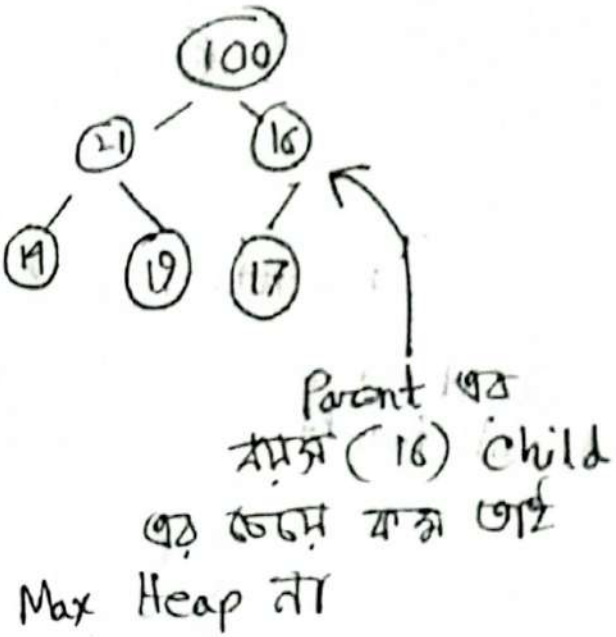
← Complete Binary tree



Heap হলো একটি Complete Binary tree

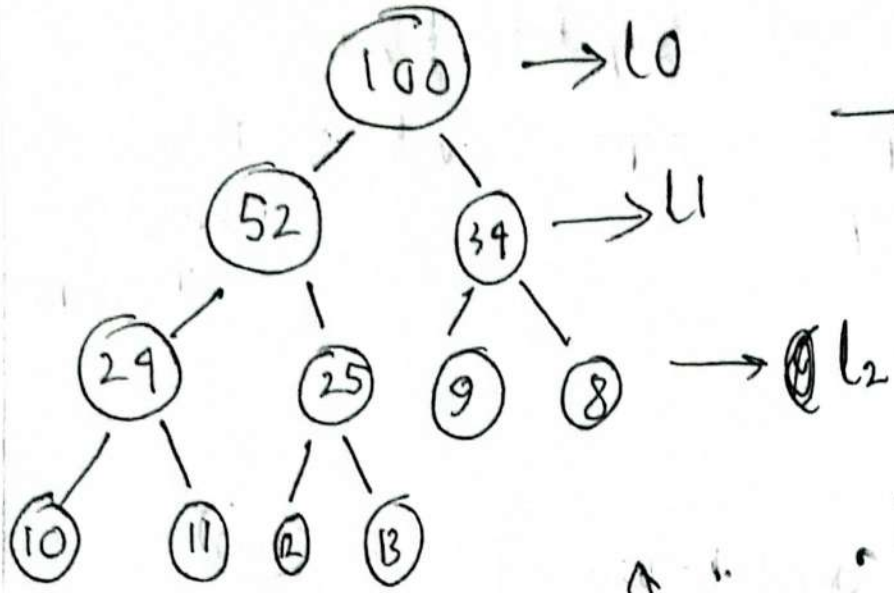
- Max heap —— যেখানে Level ও Parent এর value child এর থেকে বড় বা সমান
- Min heap





Heap Operations

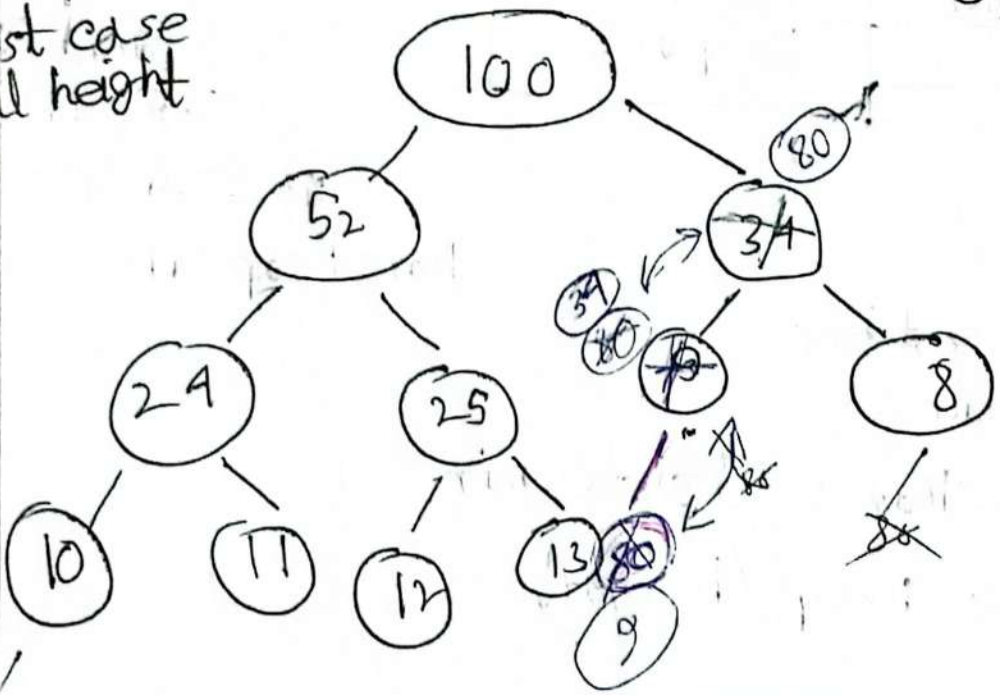
1. Insertion
2. DeleteMax or delete min
3. Make-heap / heapify



Max-heap

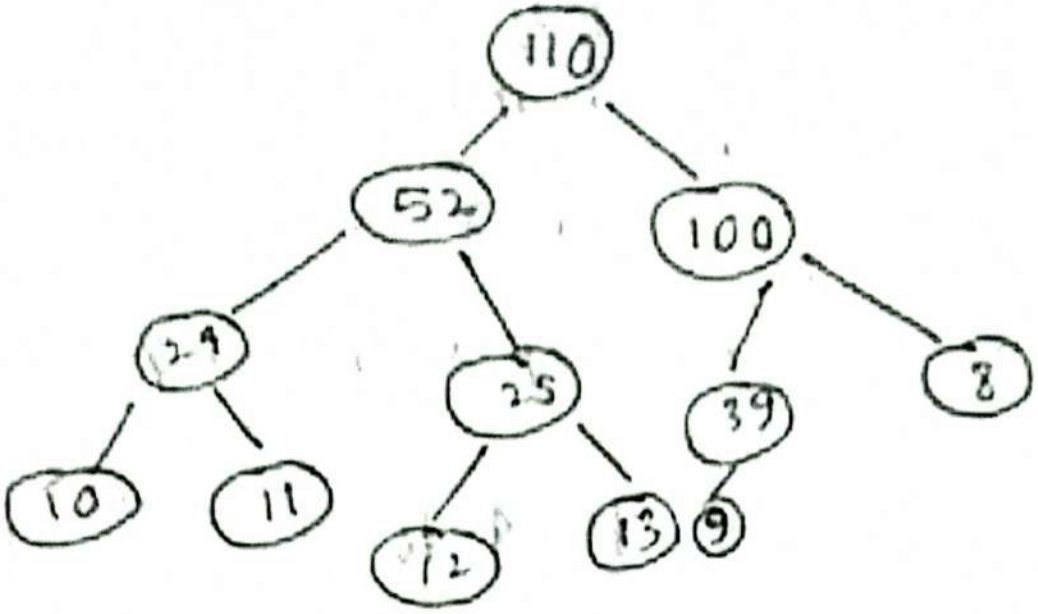
Insertion, ○ Insert, delete ও time complexity $O(\log_2 n)$

Worst case ও full height



Tree related সমস্যা আসলেই Algorithm লাগবে না। Simulation লাগবে।

Randomly যেকোন সংখ্যা নিয়ে Insert / Delete করা যায় না।



ঠা

Heapify (Heap বানানো)

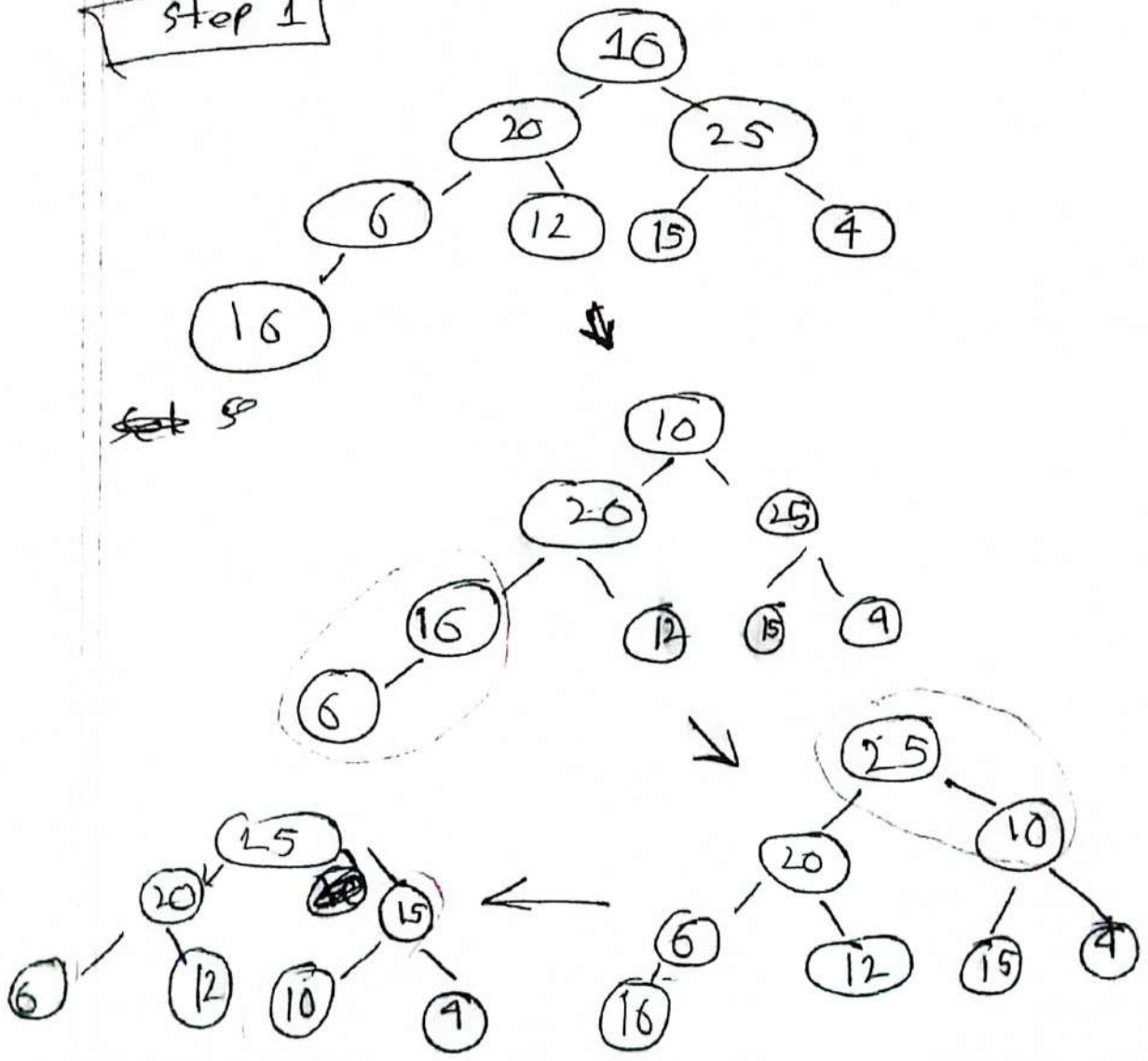
- Max heapify
- Min "

Step 1: যেই order এ আসবে সেই order এ Heap বানাবে

Step বাকিগুলো: swap - swap - swap - swap

10, 20, 25, 6, 12, 15, 4, 16

Step 1



Variant of Heap

1. Binary Heap
2. Binomial
3. Fibonacc
4. Pairing
5. Treap
6. Beap

Application

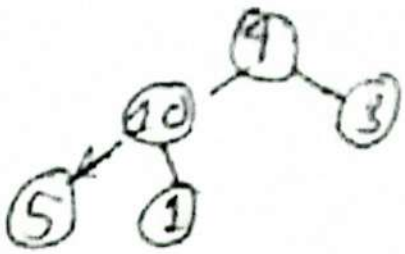
1. Heap Sort
2. Priority Q
3. Selection Algorithm
4. Graph algorithm

28/Oct/2026

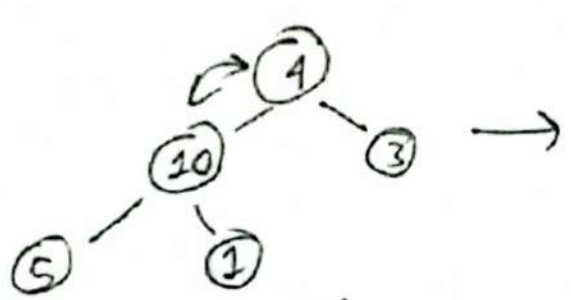
Heap.ppt (Slide 26)

Slide 27.

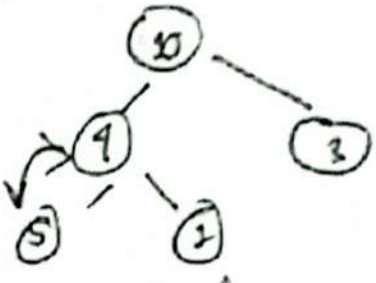
Arr = {9, 10, 3, 5, 1} →

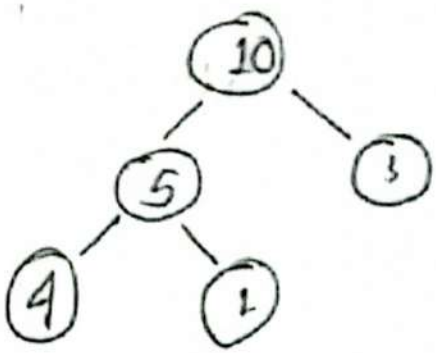


Transfer into max heap:



নিচের Level এ কোন swap প্রয়োজন হচ্ছে না

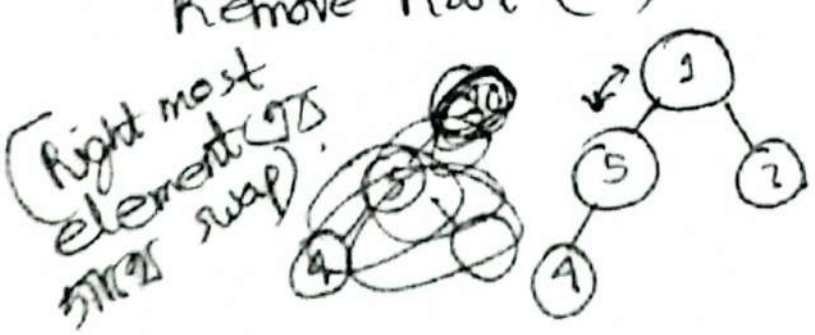


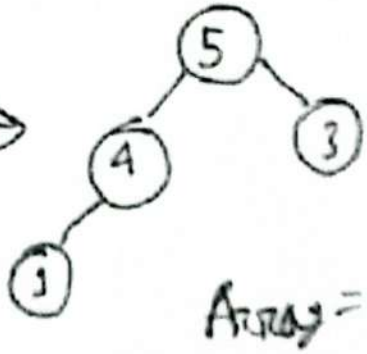


Slide 29

Remove Root (10)

(Right most element এর সাথে swap)

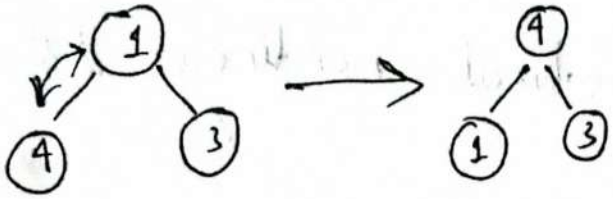




Array = {... 10}

Slide 30

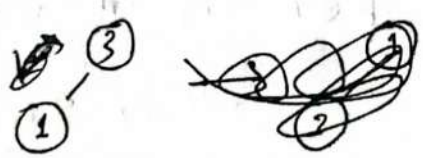
Remove Root (5)



Arr = {.... 5, 10}

Slide 31

Remove Root(4)



Arr = {....4, 5, 10}

Remove Root(3)



Arr = {3, 4, 5, 10}

Arr = {1, 3, 4, 5, 10}

time complexity $O(n \times \log_2 n)$

Heap sort Traditional গুলার চেয়ে Faster. But Counting sort এর ক্ষেত্রে Range(k) না জেনে কোনটা Faster বলা যাচ্ছে না।

Counting Sort faster হবে যদি আমরা range টুকু ভিত্তি করে।

Binary Search Tree (BST)

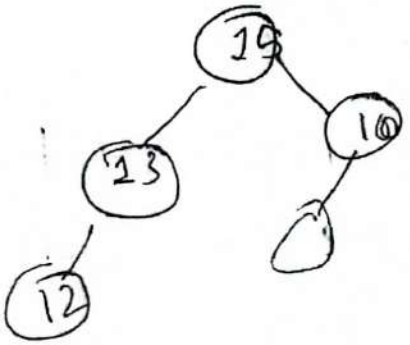
It is a hierarchical, non linear data structure.

Left child, Parent এর key ছোট হবে
Right child, Parent এর চেয়ে বড় বা সমান হবে

(সমান হলে একটু
বেশিবার
অজানা কারণে)

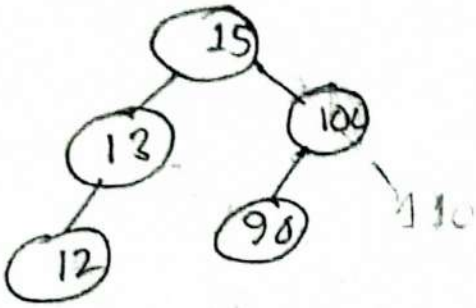
~~Search(Node, Key, Par)~~
1. If Node = NULL then return NULL
2. If Key < Node.key then ~~search(Node, key, Node)~~

Tree এর কোন Algorithm লাগাতে না লেখেও
~~Array~~ লাগাতে পারাই নাই।

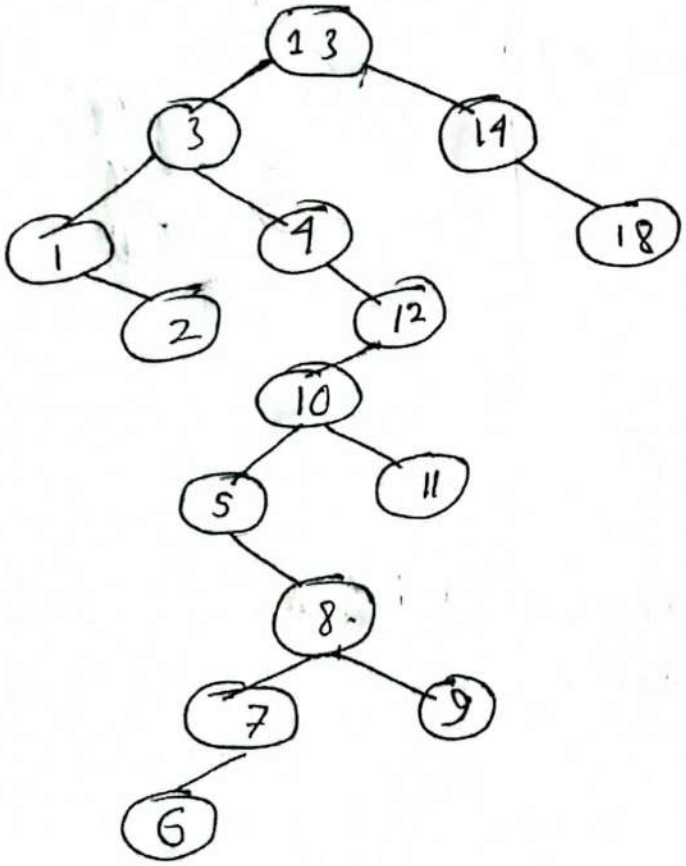


Insert

Insert (110)

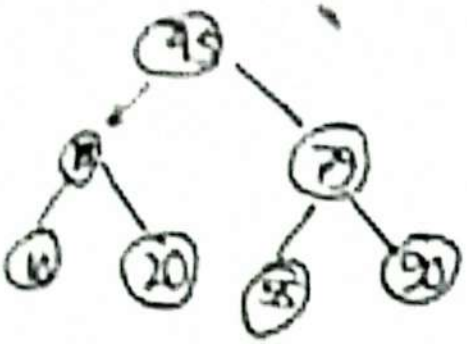


- 13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18 in that order, create a BST starting from an empty tree

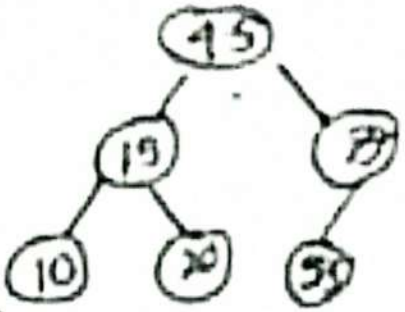


Inorder

Deleting a node from binary search tree



delete(90)

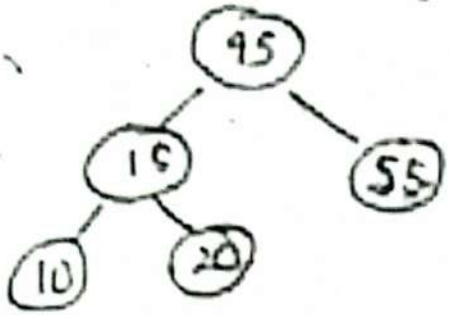


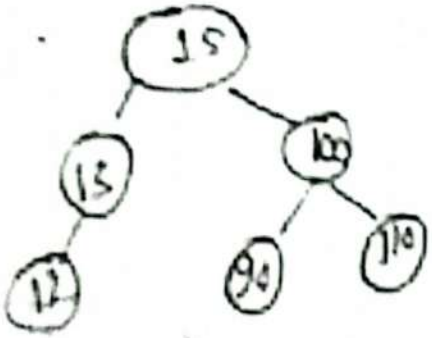
next page

delete (45)

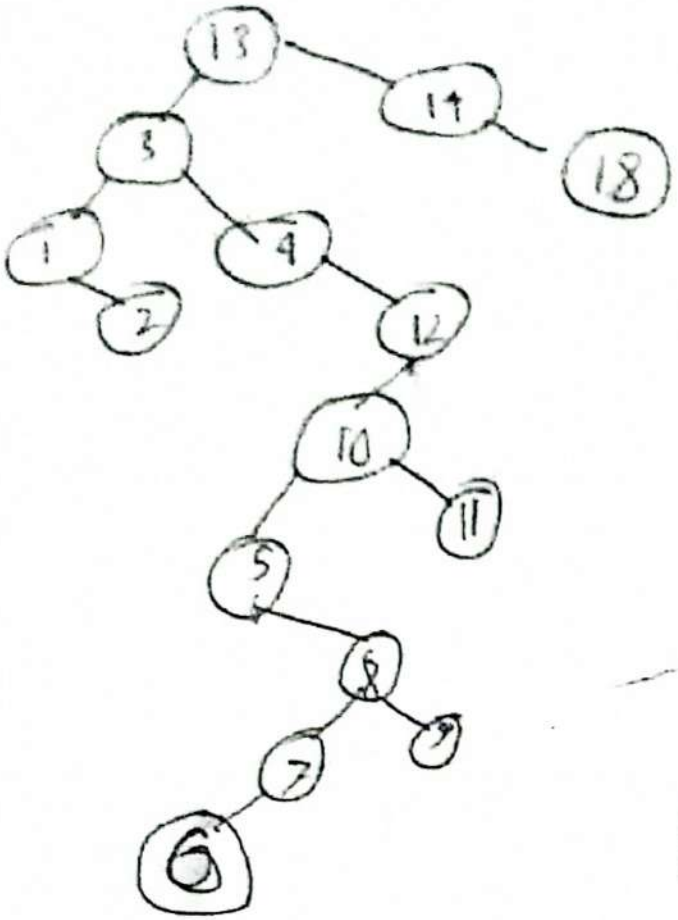
delete (79)

(Inorder successor)
(तल्ला घटकेलि)

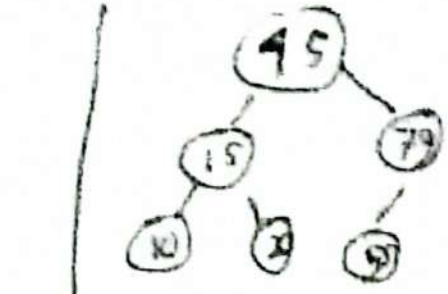




Inorder traversal : 12, 13, 1590, 100, 10
Left → Root → Right)

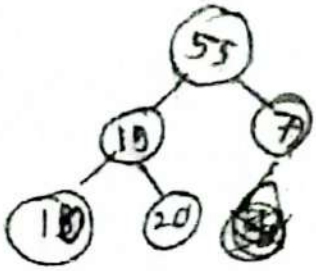


~~Inorder~~ 6, 7, 8, 9,



delete (45)

Inorder : 10, 15, 20, 45, 55, 79



Inorder: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 18

(Ascending order ও আছে)

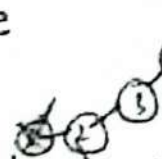
Time Complexity : $O(\log_2 n)$

↑

Insert, delete average case

delete worst case

example $O(n)$



← skewed tree

- (সবার বড় সাইজে চলে যাবে)

BST

Insert, delete, Sort (Inorder), Time complexity

Tree (৭ম) Last topic

Huffman Algorithm

1952 সালে দিয়ে ডেভিড Algorithm. (Compression)

Application:

- File compression (ZIP, GZIP)
- Image formats (JPEG, PNG)
- MP3 audio compression
- Network protocols

Slide 7

Example

apple pie

Weight

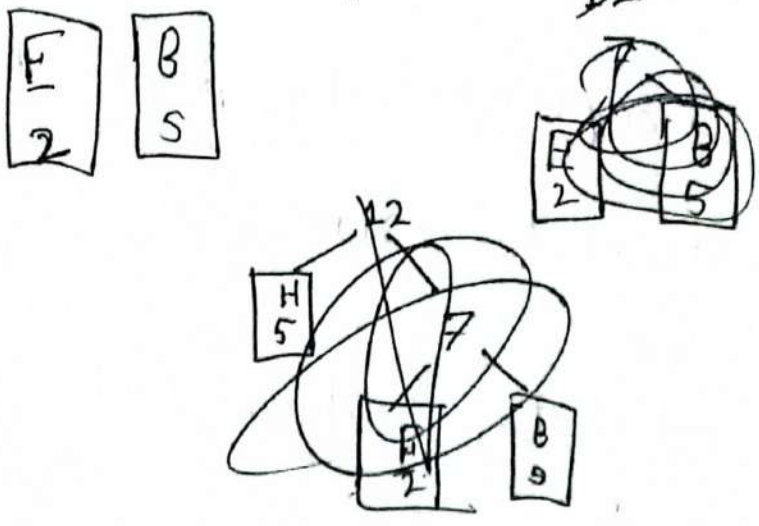
a : 1
p : 3
l : 1
e : 2
, : 1
space → □ : 1

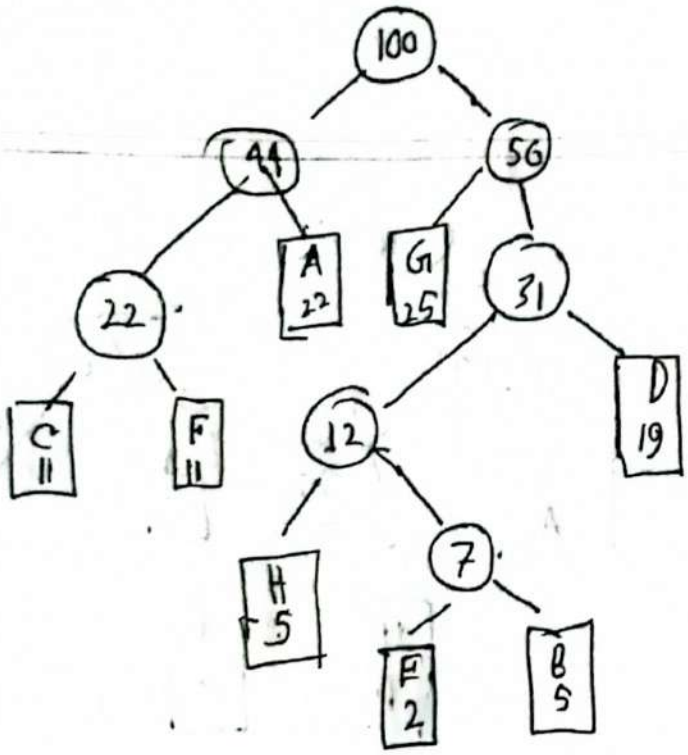
Given

A	B	C	D	E	F	G	H
22	5	11	19	2	11	25	5

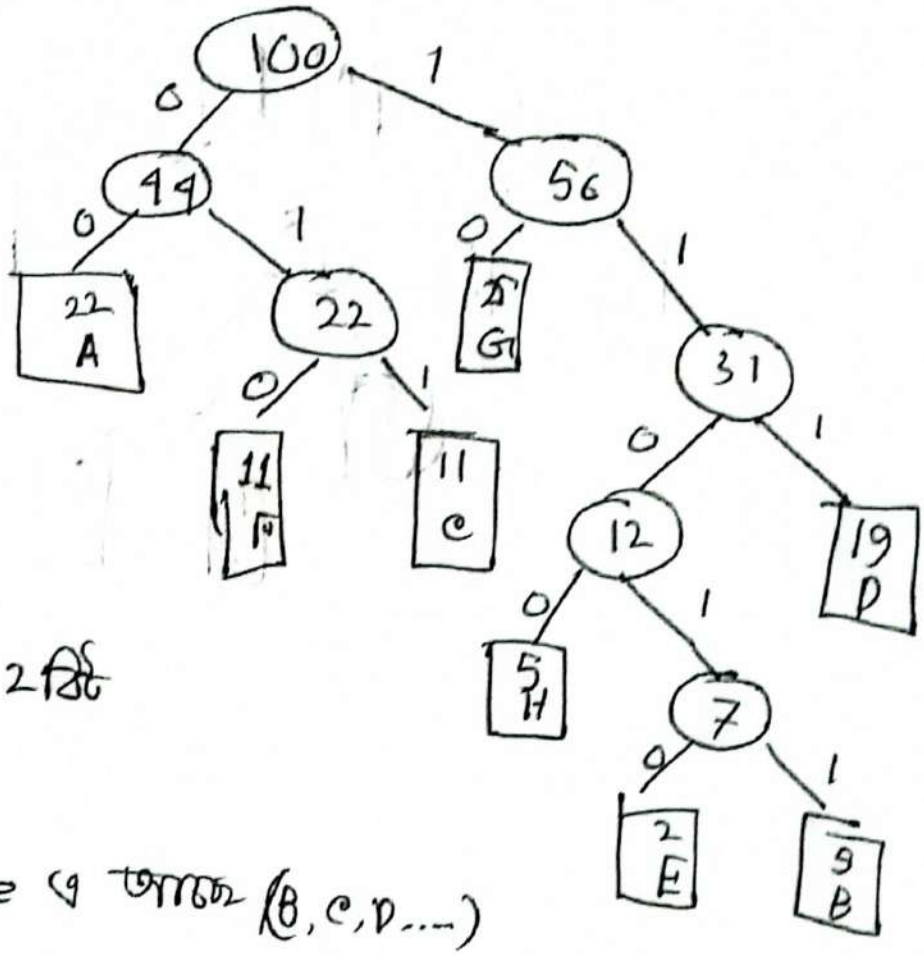
প্রথমে Ascending আকারে সাজাতে হবে

E	B	H	C	F	D	A	G
2	5	5	11	11	19	22	25




100
44
56
A
22
G
25
22
31
C
11
F
11
12
D
19
H
5
7
F
2
B
5

31/Oct/2026
online class



A : 00 ← 2 bit

F : 01 0

বাকিগুলা slide এ আছে (B, C, D....)

মাঝেজ পাঠালে ABC

প্রচলিত হয় Traditionally A = 000000 ... ← 7 bit
= 3 × 8 = 24b

A = 00 – 2 bit

B = 11011 – 5b

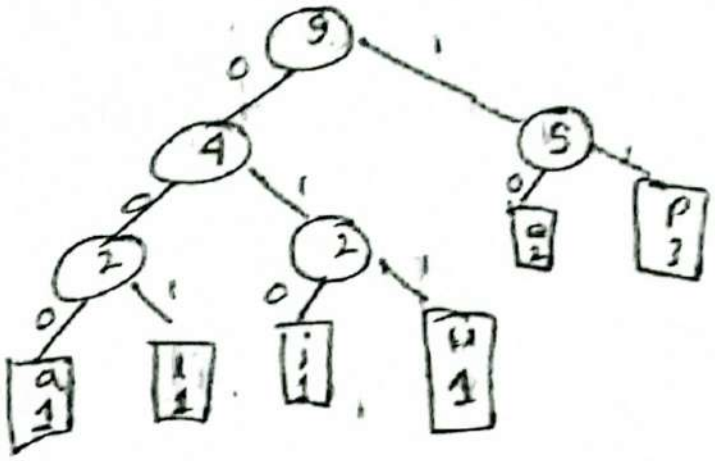
C : 0 11 – 3 b

Huffman দিয়ে পাঠালে 2 + 5 + 3 = 10 b যেই Normal 24b লাগত

Huffman Code = Variable Length Code

apple pie – Normally লাগবে $9 \times 8 = 72$ bits

Huffman দিয়ে হবে



apple pie = 000 11 11 001 10 011 11 010 10

a p p l e . p i e

(22 bits)

Size কমল $(72 - 22) = 50$ bits

Fixed Length Code (FLC)

~~Step~~ সহজ।

Huffman, FLC থেকে অনেক better
(Lossless compression technique)

Graph

A tree is a Graph with no cycle

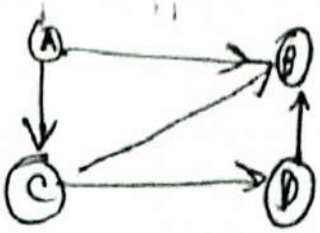
A → B → C

A → B

A → C

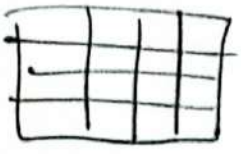
B → NULL

B → কিছু না



Dense Graph vs Sparse Graph

Adcency list Representation, Adjacency matrix



Que: Directed or undirected matrix কিভাবে বুঝবো?

Graph traversal

- Breadth-first search (BFS)
- Depth-first search (DFS)

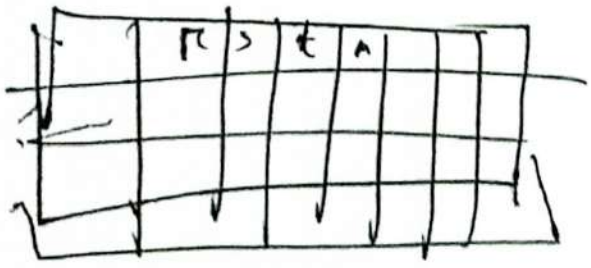
সবচুকু আজকে পড়াবে এ সম্পর্কে CT

BFS (Core Man এর বই Chapter 22)

White — have been descoved
Gray - not fully ,,
White — fully ,,

BFS Example

Slide এ দেয়া আছে



Level by Level explore করে

Q লাগবে

Time — $O(b+e)$